



INT422—Pattern Recognition
Hands-on Laboratory Exercises
for Undergraduate Machine Learning/Pattern Recognition Courses

Fall 2026

Mohammed El-Telbany



Copyright © 2026 Mohammed El-Telbany, *Associate Professor*

Published by the *Faculty of Information Technology & Computer Science*, Sinai University

<http://www.su.edu.eg>

Current as of October, 2026

Mohammed El-Telbany, Associate Professor mohammed.elsaid@su.edu.eg Cairo, 2026

LABORATORY OUTCOMES AND OBJECTIVES

This lab manual has been designed for INT422-Pattern Recognition course. This manual contains a set of lab sessions designed for INT422 students. The laboratory focuses on the basic concepts of security principle and protecting computer systems, some of the objectives and outcomes of lab sessions are:

Objectives

1. To understand basic concepts in pattern recognition.
2. To gain knowledge about state-of-the-art algorithms used in pattern recognition applications.
3. To understand pattern recognition theories, such as Bayes classifier, linear discriminant analysis.
4. To prototype, model, and test various pattern recognition algorithms.
5. To apply pattern recognition techniques in practical problems.

Outcomes

Upon successful completion of this laboratory sequence, students will be able to:

- Pre-process data, extract features, and visualize high-dimensional patterns.
- Implement Bayesian classifiers and understand decision boundaries.
- Build and evaluate distance-based, linear, and non-linear classifiers from scratch and with scikit-learn.
- Perform unsupervised learning (clustering) and dimensionality reduction (PCA, LDA).
- Train Support Vector Machines, Decision Trees, and simple Neural Networks.
- Apply rigorous evaluation methodologies (cross-validation, ROC, confusion matrices).
- Design and document a complete pattern-recognition pipeline on a real dataset.

LIST OF PRACTICAL WORK

The table below shows the typical list of practical work for *INT422—Pattern Recognition* laboratory. This list scheduling is subject to change due to unforeseen circumstances, any changes will be announced via blackboard.

Week	Lab #	Practical Work	Tools
1	Lab 1	Introduction to Pattern Recognition & Data Handling	
2	Lab 2	Bayesian Decision Theory & Generative Classifiers	
3	Lab 3	Linear Discriminants & the Perceptron	
4	Lab 4	Nearest-Neighbor Classifiers	
5	Lab 5	Unsupervised Learning — Clustering	
6	Lab 6	Dimensionality Reduction — PCA & LDA	
7	Lab 7	Support Vector Machines	
8	Lab 8	Decision Trees & Ensemble Methods	
9	Lab 9	Introduction to Neural Networks	
10	Lab 10	Model Evaluation & Selection	
11	Lab 11	Capstone — Complete Pattern Recognition Pipeline	

Required Data and Software Environment

Lab Setup Requirements

- Python 3.9 or later
- Essential libraries:


```

pip install numpy scipy matplotlib scikit-learn pandas
pip install seaborn jupyter scikit-image
      
```
- Recommended: Jupyter Notebook or VS Code for interactive experimentation
- Datasets: most experiments use built-in scikit-learn datasets (Iris, Digits, Wine, Breast Cancer). Additional data can be downloaded from UCI / OpenML when required.

LABORATORY REPORTS

During the lab time, you practice with Python tools to show how prepare the documents of the security analysis phases or workflow [4, 2, 1]. Scientists and engineers *communicate* their activities through short reports or documents. The lab report must be submitted electronically to your lab TA at the beginning of the next laboratory, late reports will not be accepted unless under extenuating circumstances.

LABORATORY SESSION AND ASSESSMENT

The attendance of the laboratory session is compulsory. If a student missed a session, he/she will be expected to make up the work in his/her own time. The INT422–Pattern Recognition is assessed in the following way:

- 10% Course work (assignments, in-class pop quizzes)
- 10% Team Project
- 20% Midterm exam
- 60% Final exam

For the Laboratory works, the students are expected to attempt a specified number of small exercises in each lab session. Normally, no allowance will be made for absence. At least four of the lab works (as assigned by the lecturer) will be assessed and counted 10% to the overall marks. Further details about the assessment will be announced by the lecturer in the class.

LAB INSTRUCTIONS

- Strictly *observe* the instructions given by the Faculty / Lab. Instructor.
- *Save* often and keep your files organized.
- *Do not* reboot, turn off, or move any workstation or PC.
- *Do not* load any software on any lab computer (without prior permission of Faculty and Technical Support Personnel). Only Teaching Assistants (TA) are authorized to carry out these tasks. *Do not* lug in external devices without scanning them for *computer viruses*.
- *Do not* reconfigure the cabling/equipment without prior permission.
- *Do not* play games on systems.
- *Turn off* the machine once you are done using it.
- *Get* the completed weekly activities/labs checked and signed by the concerned TA in the Lab.
- *Sign* weekly activities/labs are compulsory, less the half of the labs, you will not be allowed to appear in the final **Lab Examination**.

LIST OF CONDUCTED LABS

The table below shows the typical list of conducted practical work for *INT422—Pattern Recognition* laboratory with the earned degrees and TA signature.

No.	Lab #	Practical Work	Deliverable
1	Lab 1	Introduction to Pattern Recognition & Data Handling	
2	Lab 2	Bayesian Decision Theory & Generative Classifiers	
3	Lab 3	Linear Discriminants & the Perceptron	
4	Lab 4	Nearest-Neighbor Classifiers	
5	Lab 5	Unsupervised Learning — Clustering	
6	Lab 6	Dimensionality Reduction — PCA & LDA	
7	Lab 7	Support Vector Machines	
8	Lab 8	Decision Trees & Ensemble Methods	
9	Lab 9	Introduction to Neural Networks	
10	Lab 10	Model Evaluation & Selection	
11	Lab 11	Capstone — Complete Pattern Recognition Pipeline	

Table of Contents

I	Introduction	17
1	Introduction to Pattern Recognition & Data Handling	1
1.1	Objective	1
1.2	Background & Theory	1
1.2.1	Statistical Feature Scaling	2
1.2.2	Covariance Structure	2
1.3	Experiment 1.0 — Feature Extraction and Scatter Plotting	2
1.4	Experiment 1.1 — Loading and Exploring the Iris Dataset	3
1.5	Experiment 1.2 — Scatter-Plot Matrix	3
1.6	Lab Tasks	4
1.7	Assignment Questions	5
II	Statistical Learning	7
2	Bayesian Decision Theory & Generative Classifiers	9
2.1	Objective	9
2.2	Experiment 2.1 — Gaussian Naive Bayes from Scratch	10
2.3	Experiment 2.2 — Decision Region Plot	11
2.4	Assignment Questions	12
3	Linear Discriminants & the Perceptron	15
3.1	Objective	15
3.2	Experiment 3.1 — Perceptron from Scratch	16
3.3	Experiment 3.2 — Decision Boundary Animation Data	16
3.4	Assignment Questions	18
4	Nearest-Neighbor Classifiers	19
4.1	Objective	19
4.2	Experiment 4.1 — k -NN from Scratch	20
4.3	Experiment 4.2 — Decision Regions for Varying k	20
4.4	Assignment Questions	22
III	Unsupervised Learning	23
5	Unsupervised Learning — Clustering	25
5.1	Objective	25
5.2	Experiment 5.1 — k -Means from Scratch	26
5.3	Experiment 5.2 — Hierarchical Clustering Dendrogram	27
5.4	Assignment Questions	28
6	Dimensionality Reduction — PCA & LDA	29

6.1	Objective	29
6.2	Experiment 6.1 — PCA from Scratch	30
6.3	Experiment 6.2 — LDA Comparison	31
6.4	Assignment Questions	32
IV	Supervised Learning	33
7	Support Vector Machines	35
7.1	Objective	35
7.2	Experiment 7.1 — Linear SVM and Margin Visualization	35
7.3	Experiment 7.2 — RBF Kernel and Hyper-parameter Grid	36
7.4	Assignment Questions	38
8	Decision Trees & Ensemble Methods	39
8.1	Objective	39
8.2	Experiment 8.1 — Decision Tree and Feature Importance	39
8.3	Experiment 8.2 — Random Forest Comparison	40
8.4	Assignment Questions	42
9	Introduction to Neural Networks	43
9.1	Objective	43
9.2	Experiment 9.1 — MLP with scikit-learn	44
9.3	Experiment 9.2 — Learning Curves	44
9.4	Assignment Questions	46
10	Model Evaluation & Selection	47
10.1	Objective	47
10.2	Experiment 10.1 — Comprehensive Metrics	47
10.3	Experiment 10.2 — Cross-Validation Comparison	48
10.4	Assignment Questions	50
V	Capstone Project	51
11	Capstone — Complete Pattern Recognition Pipeline	53
11.1	Objective	53
11.2	Suggested Pipeline	54
11.3	Capstone Deliverables	54
11.4	Sample Report Outline	54
11.5	Assignment Questions	55
VI	Appendixes	56
A	Linux Basics	57
A.1	What is Linux?	57
A.2	Why Learn Linux?	57
A.3	Linux Architecture	57
A.4	Linux Basics Practical	57
A.4.1	Experiment A.1 — Print Working Directory	58
A.4.2	Experiment A.2 — Check Current User	58
A.4.3	Experiment A.3 — Display System Information	58

A.4.4	Experiment A.4 — Display Hostname	58
A.4.5	Experiment A.5 — Display Logged-in Users	58
A.4.6	Experiment A.6 — View Command History	58
A.4.7	Experiment A.7 — List Files and Directories	58
B	Required Python Libraries	59
C	Suggested Datasets	61
D	Further Reading	63

Part I

Introduction

LAB 1

INTRODUCTION TO PATTERN RECOGNITION & DATA HANDLING

1.1 Objective

Learning Objectives

- Understand the pattern recognition pipeline (sensing → preprocessing → feature extraction → classification → evaluation).
- Load, inspect, and visualize classic datasets.
- Perform basic pre-processing: normalization, missing-value handling, train/test split.

Pre-Lab Preparation

1. Review the concepts of **feature vector**, **class label**, and **training vs. test set** from your lecture notes.
2. Install the required libraries (`numpy`, `scikit-learn`, `matplotlib`, `seaborn`, `pandas`) and verify that `import sklearn` succeeds.
3. Read Sections 1.1–1.3 of Duda, Hart & Stork (*Pattern Classification*) or the equivalent introductory chapter of your textbook.
4. Answer briefly: What is the difference between a *feature* and a *raw measurement*? Give one example from image recognition.

1.2 Background & Theory

Pattern recognition is the scientific discipline that studies how machines can observe the environment, learn to distinguish patterns of interest, and make decisions about categories of patterns. A typical system consists of a sensor, a pre-processing stage, a feature extractor, and a classifier. Pattern recognition relies on transforming raw input into feature vectors $x \in \mathbb{R}^d$ or $x = [x_1, x_2, \dots, x_d]^T \in \mathbb{R}^d$. Visualization aids in understanding class separability, feature variance, and multimodal distributions prior to classifier design.

1.2.1 Statistical Feature Scaling

Unscaled features with disparate dynamic ranges distort distance-based classification algorithms.

$$x_{\text{min-max}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad x_{\text{z-score}} = \frac{x - \mu}{\sigma} \quad (1.1)$$

where $\mu = \frac{1}{N} \sum_{i=1}^N x_i$ and $\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$.

1.2.2 Covariance Structure

The sample covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$ captures linear dependencies:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T \quad (1.2)$$

Theory Notes

The PR Pipeline.

Raw data $\xrightarrow{\text{sensing}}$ Preprocessing $\xrightarrow{\text{feature extraction}}$ $\mathbf{x} \in \mathbb{R}^d$ $\xrightarrow{\text{classifier}}$ $\hat{y} \in \{1, \dots, C\}$

Key design choices include the representation $\mathbf{x}$, the decision rule, and the evaluation protocol.

Generalization is the ability of the classifier to perform well on unseen data; it is estimated by holding out a test set or by cross-validation. **Scaling.** Features measured in different units (e.g., cm vs. kg) can dominate distance-based or gradient-based methods. *Standardization* (zero mean, unit variance) and *min-max normalization* are the two most common remedies. Always fit scalers on the training set only and apply the same transformation to the test set.

1.3 Experiment 1.0 — Feature Extraction and Scatter Plotting

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import make_blobs
4
5  # Generate synthetic 2D feature vectors for 2 classes
6  X, y = make_blobs(n_samples=200, centers=2, n_features=2, random_state
7                    =42, cluster_std=1.5)
8
9  # Plot Feature Space
10 plt.figure(figsize=(6, 4))
11 plt.scatter(X[y==0, 0], X[y==0, 1], color='red', label='Class 0', alpha
12             =0.7)
13 plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Class 1', alpha
14             =0.7)
15 plt.xlabel('Feature $x_1$')
16 plt.ylabel('Feature $x_2$')
17 plt.title('2D Feature Space Plot')
18 plt.legend()
19 plt.grid(True)
20 plt.savefig('week1_plot.png')
21 plt.show()

```

Listing 1.1: Feature Extraction and Scatter Plotting

A 2D scatter plot showing two clusters (red and blue dots) corresponding to Class 0 and Class 1, displaying minimal overlap in the feature space x_1 vs x_2 .

1.4 Experiment 1.1 — Loading and Exploring the Iris Dataset

```

1  #!/usr/bin/env python3
2  """Week 1 - Dataset exploration and basic preprocessing."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import load_iris
7  from sklearn.model_selection import train_test_split
8  from sklearn.preprocessing import StandardScaler
9
10 # Load data
11 iris = load_iris()
12 X, y = iris.data, iris.target
13 feature_names = iris.feature_names
14 target_names = iris.target_names
15
16 print("Shape of X:", X.shape)
17 print("Classes:", target_names)
18 print("Feature names:", feature_names)
19 print("Class distribution:", np.bincount(y))
20
21 # Train/test split (stratified)
22 X_train, X_test, y_train, y_test = train_test_split(
23     X, y, test_size=0.3, random_state=42, stratify=y
24 )
25 print(f"\nTrain size: {X_train.shape[0]}, Test size: {X_test.shape[0]}")
26
27 # Standardization
28 scaler = StandardScaler()
29 X_train_scaled = scaler.fit_transform(X_train)
30 X_test_scaled = scaler.transform(X_test)
31
32 print("Mean of scaled train features (should be ~0):",
33       X_train_scaled.mean(axis=0).round(5))
34 print("Std of scaled train features (should be ~1):",
35       X_train_scaled.std(axis=0).round(5))

```

Listing 1.2: Loading and Exploring the Iris Dataset

1.5 Experiment 1.2 — Scatter-Plot Matrix

```

1  #!/usr/bin/env python3
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import load_iris
4  import pandas as pd
5  import seaborn as sns
6

```

```
7 iris = load_iris()
8 df = pd.DataFrame(iris.data, columns=iris.feature_names)
9 df['species'] = [iris.target_names[i] for i in iris.target]
10
11 sns.pairplot(df, hue='species', diag_kind='hist', height=2.2)
12 plt.suptitle("Iris Pair Plot", y=1.02)
13 plt.tight_layout()
14 plt.savefig("iris_pairplot.png", dpi=150)
15 plt.show()
```

Listing 1.3: Scatter-Plot Matrix

1.6 Lab Tasks

1. Load the Wine dataset and print shape, feature names, and class distribution.
2. Apply Standard Scaler and Min-Max Scaler; compare the resulting feature ranges.
3. Create a pair-plot (or selected 2-D scatter plots) colored by class label.
4. Why does Z-score standardization preserve Gaussian distribution shapes while Min-Max bounds features strictly to [0, 1]?
5. How does non-zero covariance affect distance calculations in Euclidean metrics?

Assignment Questions

1. **(Code)** Load the Breast Cancer Wisconsin dataset. Report the number of samples, features, and the class balance. Perform a stratified 70/30 train-test split.
2. **(Code)** Implement a function `describe_dataset(X, y)` that prints mean, std, min, max per feature and the class histogram.
3. **(Code)** Visualize the first two principal features (or any two informative features) of Iris with a scatter plot; include a legend and axis labels.
4. **(Conceptual)** Explain why stratified splitting is preferred over random splitting when classes are imbalanced. What problem can arise if stratification is omitted?

Expected Outputs

- Iris shape is (150, 4); three classes with 50 samples each.
- After StandardScaler, feature means ≈ 0 and standard deviations ≈ 1 on the training set.
- Pair-plot shows clear separation of *setosa* from the other two species on petal length/width.
- Breast Cancer dataset typically reports ~ 569 samples and 30 features with a moderate class imbalance.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Dataset loading & inspection	Correct shapes, names, class counts	20
Preprocessing (scaling + split)	Proper use of fit/transform, stratification	25
Visualization	Clear scatter/pair plot with labels & legend	25
Conceptual answer	Accurate explanation of stratification	20
Code quality & report	Readable, commented, short write-up	10
Total		100

Extra Experiment (Optional / Advanced)

Missing-value simulation & imputation. Randomly set 10% of the entries in the Iris feature matrix to NaN. Implement simple mean imputation and median imputation; compare the effect on a subsequent k -NN classifier's accuracy. Discuss when mean imputation can be harmful.

```

1  import numpy as np
2  from sklearn.impute import SimpleImputer
3  from sklearn.neighbors import KNeighborsClassifier
4  from sklearn.model_selection import cross_val_score
5  from sklearn.datasets import load_iris
6
7  X, y = load_iris(return_X_y=True)
8  rng = np.random.RandomState(0)
9  mask = rng.rand(*X.shape) < 0.10
10 X_missing = X.copy()
11 X_missing[mask] = np.nan
12
13 for strategy in ['mean', 'median']:
14     imp = SimpleImputer(strategy=strategy)
15     X_imp = imp.fit_transform(X_missing)
16     scores = cross_val_score(KNeighborsClassifier(n_neighbors=3), X_imp,
17                             y, cv=5)
18     print(f"{strategy:6s} imputation    CV accuracy = {scores.mean():.3f}")

```

1.7 Assignment Questions

Post-Lab Reflection & Discussion

1. Which two Iris features appear most discriminative from the pair-plot? Why?
2. If you had scaled the data *before* the train/test split (i.e., fitted the scaler on the whole dataset), what kind of optimistic bias would you introduce?
3. Reflect on a real-world sensor stream (e.g., wearable IMU). What preprocessing steps would be necessary before feeding the data to a classifier?
4. Submit a short paragraph (5–8 sentences) summarizing what you learned about the importance of proper data handling.

Part II

Statistical Learning

LAB 2

BAYESIAN DECISION THEORY & GENERATIVE CLASSIFIERS

2.1 Objective

Learning Objectives

- Implement a Gaussian Naive Bayes classifier from first principles.
- Visualize decision regions in 2D feature space.
- Compare empirical error with the Bayes error estimate.

Pre-Lab Preparation

1. Review Bayes' theorem and the definition of the maximum-a-posteriori (MAP) decision rule.
2. Recall the univariate Gaussian density formula. Write it down without looking it up.
3. What does the “naive” assumption in Naive Bayes state? Give a one-sentence answer.
4. Ensure you can compute a confusion matrix by hand for a small 2-class example.

Theory Notes

Bayes Decision Theory. Given feature vector $\mathbf{x}$ and class-conditional densities $p(\mathbf{x} | y = c)$ together with priors $P(y = c)$, the posterior is

$$P(y = c | \mathbf{x}) = \frac{p(\mathbf{x} | y = c) P(y = c)}{p(\mathbf{x})}.$$

The Bayes classifier chooses $\hat{y} = \arg \max_c P(y = c | \mathbf{x})$. Under 0-1 loss this rule minimises the probability of error.

Gaussian Naive Bayes. Assume class-conditional independence of features and univariate Gaussian margins:

$$p(\mathbf{x} | y = c) = \prod_{j=1}^d \frac{1}{\sqrt{2\pi\sigma_{c,j}^2}} \exp\left(-\frac{(x_j - \mu_{c,j})^2}{2\sigma_{c,j}^2}\right).$$

Parameters $\mu_{c,j}$, $\sigma_{c,j}^2$ and priors are estimated by maximum likelihood on the training set. The deci-

sion boundary between two classes is in general a quadratic curve (or a hyperplane when variances are tied).

2.2 Experiment 2.1 — Gaussian Naive Bayes from Scratch

```
1  #!/usr/bin/env python3
2  """Week 2 - Gaussian Naive Bayes classifier."""
3
4  import numpy as np
5  from sklearn.datasets import load_iris
6  from sklearn.model_selection import train_test_split
7  from sklearn.metrics import accuracy_score, confusion_matrix
8
9  class GaussianNBScratch:
10 def fit(self, X, y):
11     self.classes_ = np.unique(y)
12     self.mean_ = {}
13     self.var_ = {}
14     self.priors_ = {}
15     for c in self.classes_:
16         X_c = X[y == c]
17         self.mean_[c] = X_c.mean(axis=0)
18         self.var_[c] = X_c.var(axis=0) + 1e-9 # avoid div-by-zero
19         self.priors_[c] = X_c.shape[0] / X.shape[0]
20     return self
21
22 def _pdf(self, x, mean, var):
23     return np.exp(-0.5 * (x - mean)**2 / var) / np.sqrt(2 * np.pi * var)
24
25 def predict_proba(self, X):
26     posteriors = []
27     for x in X:
28         scores = []
29         for c in self.classes_:
30             prior = np.log(self.priors_[c])
31             likelihood = np.sum(np.log(self._pdf(x, self.mean_[c], self.var_[c])))
32             scores.append(prior + likelihood)
33         # softmax for numerical stability
34         scores = np.array(scores)
35         scores -= scores.max()
36         exp_s = np.exp(scores)
37         posteriors.append(exp_s / exp_s.sum())
38     return np.array(posteriors)
39
40 def predict(self, X):
41     return self.classes_[np.argmax(self.predict_proba(X), axis=1)]
42
43 # Demo
44 iris = load_iris()
45 X, y = iris.data[:, :2], iris.target # use first two features
46     for viz
47 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
48     random_state=0, stratify=y)
49
50 clf = GaussianNBScratch().fit(X_tr, y_tr)
51 y_pred = clf.predict(X_te)
```

```

50 print("Accuracy:", accuracy_score(y_te, y_pred).round(3))
51 print("Confusion matrix:\n", confusion_matrix(y_te, y_pred))

```

Listing 2.1: Gaussian Naive Bayes from Scratch

2.3 Experiment 2.2 — Decision Region Plot

```

1  #!/usr/bin/env python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import load_iris
5  from sklearn.naive_bayes import GaussianNB
6  from sklearn.model_selection import train_test_split
7
8  iris = load_iris()
9  X, y = iris.data[:, :2], iris.target
10 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
11     random_state=0)
12
13 clf = GaussianNB().fit(X_tr, y_tr)
14
15 # Mesh grid
16 x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
17 y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
18 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
19     np.linspace(y_min, y_max, 200))
20 Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
21
22 plt.figure(figsize=(8, 6))
23 plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Set1)
24 plt.scatter(X_tr[:, 0], X_tr[:, 1], c=y_tr, edgecolor='k', cmap=plt.cm.
25     Set1)
26 plt.xlabel(iris.feature_names[0])
27 plt.ylabel(iris.feature_names[1])
28 plt.title("Gaussian NB Decision Regions (Iris, 2 features)")
29 plt.savefig("nb_decision_regions.png", dpi=150)
30 plt.show()

```

Listing 2.2: Decision Region Plot

Assignment Questions

1. **(Code)** Implement Gaussian Naive Bayes from scratch (as above) and compare its accuracy on Iris (all 4 features) with `sklearn.naive_bayes.GaussianNB`.
2. **(Code)** Plot 2-D decision regions for the first two features of the Wine dataset using both your implementation and scikit-learn.
3. **(Code)** Compute the empirical class-conditional means and variances; verify they match the attributes stored inside the fitted scikit-learn model.
4. **(Conceptual)** Under what assumptions is the Naive Bayes classifier optimal? What happens when features are strongly correlated?

Expected Outputs

- Scratch and sklearn GaussianNB accuracies on Iris (4 features) should be nearly identical (typically > 0.95).
- Decision-region plot shows three coloured regions separated by smooth curved boundaries.
- Printed means/variances match `clf.theta_` and `clf.var_` of the sklearn estimator (within floating-point tolerance).

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Scratch NB implementation	Correct fit/predict, numerical stability	30
Accuracy comparison	Matches sklearn within 1–2%	20
Decision-region visualization	Clear plot with training points overlaid	25
Conceptual answer	Correct statement of NB assumptions	15
Code quality	Clean, commented, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

Quadratic Discriminant Analysis (QDA) comparison. Train both GaussianNB (shared diagonal covariance) and QDA (full per-class covariance) on the first two features of the Wine dataset. Plot both decision regions side-by-side and compare test accuracy. Discuss when the extra flexibility of QDA helps or hurts.

```

1  from sklearn.discriminant_analysis import
    QuadraticDiscriminantAnalysis
2  from sklearn.naive_bayes import GaussianNB
3  from sklearn.datasets import load_wine
4  from sklearn.model_selection import train_test_split
5
6  X, y = load_wine(return_X_y=True)
7  X = X[:, :2]
8  X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
    random_state=0, stratify=y)
9  for name, clf in [('NB', GaussianNB()), ('QDA',
    QuadraticDiscriminantAnalysis())]:
10     clf.fit(X_tr, y_tr)
11     print(name, "test accuracy:", round(clf.score(X_te, y_te), 3))
12

```

2.4 Assignment Questions**Post-Lab Reflection & Discussion**

1. Did your scratch implementation match sklearn? If not, where did the numerical difference arise?
2. Sketch (by hand) the decision boundary that would result if all class-conditional variances

were forced to be identical.

3. In a medical diagnosis setting, which is more costly: a false positive or a false negative? How would you modify the Bayes decision rule to reflect asymmetric costs?
4. Write a short reflection on the practical value (and limitations) of the independence assumption.

LAB 3

LINEAR DISCRIMINANTS & THE PERCEPTRON

3.1 Objective

Learning Objectives

- Implement the Perceptron learning algorithm.
- Visualize the evolution of the decision boundary.
- Understand linear separability and the limitations of a single-layer perceptron.

Pre-Lab Preparation

1. Review the definition of a linear decision boundary: $\mathbf{w}^\top \mathbf{x} + b = 0$.
2. What does it mean for two sets of points to be *linearly separable*?
3. Write the Perceptron update rule from memory (or from the lecture slides).
4. Generate (mentally or on paper) a tiny 2-D example with 4 points that is linearly separable and one that is not.

Theory Notes

The Perceptron Criterion. For a labelled sample $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$, the Perceptron aims to find $\mathbf{w}, b$ such that

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0 \quad \forall i.$$

The stochastic update is

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \quad b \leftarrow b + \eta y_i$$

whenever the i -th example is misclassified. **Novikoff's theorem** guarantees that, for linearly separable data with margin γ and bounded $\|\mathbf{x}\| \leq R$, the algorithm makes at most $(R/\gamma)^2$ updates.

A single-layer Perceptron cannot solve non-linearly separable problems (e.g., XOR). Multi-layer networks or kernel methods are required.

3.2 Experiment 3.1 — Perceptron from Scratch

```
1  #!/usr/bin/env python3
2  """Week 3 - Classic Perceptron algorithm."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_classification
7
8  class Perceptron:
9  def __init__(self, lr=0.01, n_epochs=100):
10 self.lr = lr
11 self.n_epochs = n_epochs
12
13 def fit(self, X, y):
14 # map labels to {-1, +1}
15 y_ = np.where(y == 0, -1, 1)
16 n_samples, n_features = X.shape
17 self.w_ = np.zeros(n_features)
18 self.b_ = 0.0
19 self.errors_ = []
20
21 for _ in range(self.n_epochs):
22 errors = 0
23 for xi, target in zip(X, y_):
24 update = self.lr * (target - self.predict_raw(xi))
25 if update != 0:
26 self.w_ += update * xi
27 self.b_ += update
28 errors += 1
29 self.errors_.append(errors)
30 if errors == 0:
31 break
32 return self
33
34 def predict_raw(self, X):
35 return np.dot(X, self.w_) + self.b_
36
37 def predict(self, X):
38 return np.where(self.predict_raw(X) >= 0, 1, 0)
39
40 # Linearly separable synthetic data
41 X, y = make_classification(n_samples=100, n_features=2, n_redundant=0,
42 n_informative=2, n_clusters_per_class=1,
43 class_sep=1.5, random_state=42)
44 # Make it strictly 2-class and roughly separable
45 y = y % 2
46
47 ppn = Perceptron(lr=0.1, n_epochs=50).fit(X, y)
48 print("Final weights:", ppn.w_.round(3), "bias:", round(ppn.b_, 3))
49 print("Misclassifications per epoch:", ppn.errors_)
```

Listing 3.1: Perceptron from Scratch

3.3 Experiment 3.2 — Decision Boundary Animation Data

```

1  #!/usr/bin/env python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import make_classification
5
6  X, y = make_classification(n_samples=80, n_features=2, n_redundant=0,
7  n_informative=2, n_clusters_per_class=1,
8  class_sep=2.0, random_state=1)
9  y = y % 2
10
11 def plot_boundary(w, b, X, y, title=""):
12     x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
13     xx = np.linspace(x_min, x_max, 200)
14     # w0*x + w1*y + b = 0 => y = (-w0*x - b)/w1
15     if abs(w[1]) > 1e-9:
16         yy = (-w[0] * xx - b) / w[1]
17         plt.plot(xx, yy, 'k-', lw=2)
18         plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolor='k')
19         plt.title(title)
20         plt.xlabel("x1"); plt.ylabel("x2")
21
22 # After training (reuse Perceptron class from above)
23 ppn = Perceptron(lr=0.05, n_epochs=30).fit(X, y)
24 plt.figure(figsize=(7, 5))
25 plot_boundary(ppn.w_, ppn.b_, X, y, "Perceptron Decision Boundary")
26 plt.savefig("perceptron_boundary.png", dpi=150)
27 plt.show()

```

Listing 3.2: Decision Boundary Animation Data

Assignment Questions

1. **(Code)** Implement the Perceptron and train it on a linearly separable synthetic dataset. Plot the number of misclassifications versus epoch.
2. **(Code)** Visualize the final decision boundary together with the training points.
3. **(Code)** Apply the same Perceptron to a non-separable dataset (reduce `class_sep`). Report whether the algorithm converges and the final training error.
4. **(Conceptual)** Prove (or argue) that the Perceptron converges in a finite number of steps when the data are linearly separable. What is the role of the learning rate?

Expected Outputs

- On a well-separated dataset the error curve reaches zero within a few dozen epochs.
- Decision boundary is a straight line that correctly separates the two classes.
- On non-separable data the error curve oscillates and never reaches zero; training accuracy stays below 100%.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Perceptron implementation	Correct weight update, convergence check	30
Error curve & boundary plot	Clear figures with labels	25
Non-separable experiment	Correct observation of non-convergence	20
Conceptual answer	Sound argument about convergence	15
Code quality	Clean, reproducible experiments	10
Total		100

Extra Experiment (Optional / Advanced)

Pocket algorithm for non-separable data. Modify the Perceptron so that it keeps the weight vector that achieved the fewest training errors so far (the “pocket”). Run it on a non-separable dataset for a fixed number of epochs and compare the pocket solution with the final weights.

```
1  # Pseudocode extension inside the training loop:
2  # if errors < best_errors:
3  #     best_w, best_b = w.copy(), b
4  #     best_errors = errors
5  # After training return best_w, best_b instead of final w, b
6
```

3.4 Assignment Questions

Post-Lab Reflection & Discussion

1. Did the Perceptron converge on your separable dataset? How many epochs were required?
2. What happens to the decision boundary if you multiply all weights by a positive constant after training?
3. Why can the classic Perceptron oscillate forever on non-separable data? How does the pocket algorithm mitigate this?
4. Reflect on a real problem (e.g., spam detection) that is almost linearly separable. Would you still prefer a Perceptron or a more flexible model?

LAB 4

NEAREST-NEIGHBOR CLASSIFIERS

4.1 Objective

Learning Objectives

- Implement k -Nearest Neighbors from scratch.
- Study the effect of k and distance metric on accuracy.
- Visualize decision regions for different values of k .

Pre-Lab Preparation

1. Recall the definitions of Euclidean and Manhattan distances in $\mathbb{R}^d$.
2. What is the computational complexity of a naive k -NN query for N training points in d dimensions?
3. Sketch how the decision boundary of 1-NN relates to the Voronoi diagram of the training set.
4. Why is feature scaling especially important for distance-based methods?

Theory Notes

k -Nearest Neighbours. Given a query $\mathbf{x}$, find the k training points closest to $\mathbf{x}$ under a metric $d(\cdot, \cdot)$ and predict by majority vote (classification) or average (regression). As $N \rightarrow \infty$ and $k \rightarrow \infty$ with $k/N \rightarrow 0$, the k -NN risk approaches the Bayes risk.

Bias-variance trade-off.

- $k = 1$: low bias, high variance (sensitive to noise and outliers).
- Large k : higher bias, lower variance (smoother boundary).

The optimal k is data-dependent and is usually selected by cross-validation. Curse of dimensionality: in high dimensions distances concentrate and the notion of “nearest” becomes less meaningful.

4.2 Experiment 4.1 — k -NN from Scratch

```
1  #!/usr/bin/env python3
2  """Week 4 - k-Nearest Neighbours classifier."""
3
4  import numpy as np
5  from collections import Counter
6  from sklearn.datasets import load_iris
7  from sklearn.model_selection import train_test_split
8  from sklearn.metrics import accuracy_score
9
10 class KNN:
11     def __init__(self, k=3, metric='euclidean'):
12         self.k = k
13         self.metric = metric
14
15     def fit(self, X, y):
16         self.X_train = np.asarray(X)
17         self.y_train = np.asarray(y)
18         return self
19
20     def _distance(self, a, b):
21         if self.metric == 'euclidean':
22             return np.sqrt(np.sum((a - b)**2, axis=1))
23         elif self.metric == 'manhattan':
24             return np.sum(np.abs(a - b), axis=1)
25         else:
26             raise ValueError("Unknown metric")
27
28     def predict(self, X):
29         X = np.asarray(X)
30         preds = []
31         for x in X:
32             dists = self._distance(self.X_train, x)
33             idx = np.argsort(dists)[:self.k]
34             votes = self.y_train[idx]
35             preds.append(Counter(votes).most_common(1)[0][0])
36         return np.array(preds)
37
38     iris = load_iris()
39     X, y = iris.data, iris.target
40     X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
41                                               random_state=42, stratify=y)
42
43     for k in [1, 3, 5, 11]:
44         clf = KNN(k=k).fit(X_tr, y_tr)
45         acc = accuracy_score(y_te, clf.predict(X_te))
46         print(f"k={k:2d}  accuracy={acc:.3f}")
```

Listing 4.1: k -NN from Scratch

4.3 Experiment 4.2 — Decision Regions for Varying k

```
1  #!/usr/bin/env python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import load_iris
```

```

5  from sklearn.neighbors import KNeighborsClassifier
6  from sklearn.model_selection import train_test_split
7
8  iris = load_iris()
9  X, y = iris.data[:, :2], iris.target
10 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
11     random_state=0)
12
13 fig, axes = plt.subplots(1, 3, figsize=(14, 4))
14 for ax, k in zip(axes, [1, 5, 15]):
15     clf = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
16     x_min, x_max = X[:, 0].min()-0.5, X[:, 0].max()+0.5
17     y_min, y_max = X[:, 1].min()-0.5, X[:, 1].max()+0.5
18     xx, yy = np.meshgrid(np.linspace(x_min, x_max, 150),
19         np.linspace(y_min, y_max, 150))
20     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
21     ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Set1)
22     ax.scatter(X_tr[:, 0], X_tr[:, 1], c=y_tr, edgecolor='k', cmap=plt.cm.
23         Set1, s=30)
24     ax.set_title(f"k = {k}")
25     plt.tight_layout()
26     plt.savefig("knn_regions.png", dpi=150)
27     plt.show()

```

Listing 4.2: Decision Regions for Varying k

Assignment Questions

1. **(Code)** Implement k -NN from scratch supporting both Euclidean and Manhattan distances. Evaluate on Iris for $k \in \{1, 3, 5, 11\}$.
2. **(Code)** Plot decision regions for $k = 1$, $k = 5$ and $k = 15$ on the first two features of Iris. Comment on the smoothness of the boundary.
3. **(Code)** Use cross-validation (or a validation set) to select a good value of k for the Digits dataset (or a subset of it).
4. **(Conceptual)** Explain the bias-variance trade-off as k increases. What happens at the extremes $k = 1$ and $k = N$?

Expected Outputs

- Accuracy on Iris is typically high (> 0.90) for moderate k ; $k = 1$ may overfit slightly.
- Decision regions for $k = 1$ are highly irregular; for larger k the boundaries become smoother.
- Cross-validation curve shows a peak at an intermediate k and a decline for very large k .

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Scratch k -NN implementation	Correct distances, voting, both metrics	30
Decision-region plots	Three clear figures, comment on smoothness	25
Model selection for k	Sensible CV procedure and chosen k	20
Conceptual answer	Correct bias-variance discussion	15
Code quality	Clean, efficient enough for the datasets	10
Total		100

Extra Experiment (Optional / Advanced)

Weighted k -NN & distance metrics. Implement a distance-weighted k -NN (inverse-distance voting) and compare it with uniform voting on the Digits dataset. Also evaluate the Mahalanobis distance (using the training-set covariance) versus Euclidean distance.

```
1 # Weighted vote example (inside predict):  
2 # weights = 1.0 / (dists[idx] + 1e-9)  
3 # Choose class with largest sum of weights  
4
```

4.4 Assignment Questions

Post-Lab Reflection & Discussion

1. For which value of k did you obtain the highest accuracy on Iris? Does the same k work best on Digits?
2. Why does the decision boundary become smoother as k grows?
3. Discuss one practical limitation of storing the entire training set (memory, latency) and a possible mitigation (e.g., condensed NN, KD-trees).
4. Reflect: in which application domains is k -NN still competitive despite its simplicity?

Part III

Unsupervised Learning

LAB 5

UNSUPERVISED LEARNING — CLUSTERING

5.1 Objective

Learning Objectives

- Implement k -means clustering from scratch.
- Evaluate clusters with inertia, silhouette score and purity.
- Apply hierarchical clustering and visualize dendrograms.

Pre-Lab Preparation

1. What is the objective function (inertia / within-cluster sum of squares) that k -means minimises?
2. Why is the result of k -means sensitive to the initial placement of centroids?
3. Briefly describe the difference between partitional and hierarchical clustering.
4. Look up the definition of the silhouette coefficient; write the formula for a single sample.

Theory Notes

k -Means. Alternate between (i) assigning each point to the nearest centroid and (ii) updating each centroid to the mean of its assigned points. The algorithm converges to a local minimum of

$$J = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2.$$

Hierarchical clustering builds a tree (dendrogram) by successively merging (agglomerative) or splitting (divisive) clusters. Ward linkage minimises the increase in total within-cluster variance at each merge.

Evaluation without labels relies on internal indices (silhouette, Davies-Bouldin); with labels one may compute purity or adjusted Rand index.

5.2 Experiment 5.1 — k -Means from Scratch

```
1  #!/usr/bin/env python3
2  """Week 5 - k-Means clustering."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_blobs
7  from sklearn.metrics import silhouette_score
8
9  class KMeansScratch:
10 def __init__(self, n_clusters=3, max_iter=100, random_state=42):
11     self.n_clusters = n_clusters
12     self.max_iter = max_iter
13     self.random_state = random_state
14
15 def fit(self, X):
16     rng = np.random.RandomState(self.random_state)
17     # random init
18     idx = rng.choice(X.shape[0], self.n_clusters, replace=False)
19     self.centroids_ = X[idx].copy()
20     for i in range(self.max_iter):
21         # assignment step
22         dists = np.linalg.norm(X[:, None, :] - self.centroids_[None, :, :], axis
23                                 =2)
24         labels = np.argmin(dists, axis=1)
25         # update step
26         new_centroids = np.array([X[labels == k].mean(axis=0)
27                                   for k in range(self.n_clusters)])
28         if np.allclose(self.centroids_, new_centroids):
29             break
30         self.centroids_ = new_centroids
31         self.labels_ = labels
32     return self
33
34 def predict(self, X):
35     dists = np.linalg.norm(X[:, None, :] - self.centroids_[None, :, :], axis
36                             =2)
37     return np.argmin(dists, axis=1)
38
39 # Synthetic blobs
40 X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.7,
41                        random_state=42)
42 km = KMeansScratch(n_clusters=4).fit(X)
43 print("Silhouette score:", round(silhouette_score(X, km.labels_), 3))
44
45 plt.figure(figsize=(7, 5))
46 plt.scatter(X[:, 0], X[:, 1], c=km.labels_, cmap='viridis', s=30,
47             edgecolor='k')
48 plt.scatter(km.centroids_[0, 0], km.centroids_[0, 1], c='red', marker='x',
49             s=200, label='centroids')
50 plt.legend(); plt.title("k-Means (scratch)")
51 plt.savefig("kmeans_scratch.png", dpi=150)
52 plt.show()
```

Listing 5.1: k -Means from Scratch

5.3 Experiment 5.2 — Hierarchical Clustering Dendrogram

```

1  #!/usr/bin/env python3
2  import matplotlib.pyplot as plt
3  from scipy.cluster.hierarchy import dendrogram, linkage
4  from sklearn.datasets import load_iris
5
6  iris = load_iris()
7  X = iris.data
8
9  Z = linkage(X, method='ward')
10 plt.figure(figsize=(12, 5))
11 dendrogram(Z, truncate_mode='lastp', p=20, leaf_rotation=90)
12 plt.title("Hierarchical Clustering Dendrogram (Ward)")
13 plt.xlabel("Sample index or (cluster size)")
14 plt.ylabel("Distance")
15 plt.tight_layout()
16 plt.savefig("dendrogram.png", dpi=150)
17 plt.show()

```

Listing 5.2: Hierarchical Clustering Dendrogram

Assignment Questions

1. **(Code)** Implement k -means from scratch. Run it on a synthetic 4-blob dataset and on the Iris data (without using labels). Report inertia and silhouette score.
2. **(Code)** Plot the clustering result with centroids marked. Experiment with different random initializations.
3. **(Code)** Produce a dendrogram for the Iris dataset using Ward linkage. Cut the tree to obtain 3 clusters and compare with the true labels (purity or adjusted Rand index).
4. **(Conceptual)** Explain the difference between k -means and hierarchical clustering. When would you prefer one over the other?

Expected Outputs

- On well-separated blobs, silhouette score is typically > 0.6 ; centroids sit near the true centres.
- Iris k -means with $k = 3$ recovers the three species reasonably well (especially setosa).
- Dendrogram shows a clear hierarchy; cutting at the appropriate height yields three clusters.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
k -means implementation	Correct assignment & update steps	30
Evaluation metrics & plots	Silhouette, inertia, clear visualization	25
Hierarchical clustering	Correct dendrogram and cut comparison	20
Conceptual answer	Clear comparison of the two paradigms	15
Code quality	Clean, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

Choosing K with the elbow method & silhouette. Run k -means for $K = 2 \dots 10$ on a synthetic dataset with a known number of blobs. Plot inertia and mean silhouette versus K . Identify the “elbow” and the K that maximises silhouette; discuss whether they agree.

```
1 inertias, silhouettes = [], []
2 for k in range(2, 11):
3     km = KMeansScratch(n_clusters=k, random_state=0).fit(X)
4     inertias.append(np.sum((X - km.centroids_[km.labels_])**2))
5     silhouettes.append(silhouette_score(X, km.labels_))
6     # plot both curves
7
```

5.4 Assignment Questions

Post-Lab Reflection & Discussion

1. Did different random initializations produce different final clusterings on Iris? By how much did the silhouette score vary?
2. Why does Ward linkage tend to produce more equally-sized clusters than single linkage?
3. Give a concrete example of a dataset where k -means would fail (e.g., non-convex clusters) and suggest an alternative algorithm.
4. Reflect on an application (customer segmentation, image compression, ...) where clustering is more natural than supervised classification.

LAB 6

DIMENSIONALITY REDUCTION — PCA & LDA

6.1 Objective

Learning Objectives

- Implement Principal Component Analysis from scratch via eigendecomposition.
- Apply Linear Discriminant Analysis for supervised projection.
- Visualize high-dimensional data in 2-D and quantify reconstruction error.

Pre-Lab Preparation

1. Review eigenvalues and eigenvectors of a symmetric matrix. What does the eigendecomposition of the covariance matrix represent geometrically?
2. Write the reconstruction formula of PCA: $\hat{\mathbf{x}} = \boldsymbol{\mu} + \sum_{i=1}^m z_i \mathbf{u}_i$.
3. What is the difference between *unsupervised* (PCA) and *supervised* (LDA) dimensionality reduction?
4. Why must data be centred (and preferably scaled) before computing PCA?

Theory Notes

Principal Component Analysis. PCA finds an orthonormal basis $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_d]$ that maximises captured variance. The principal directions are the eigenvectors of the sample covariance

$$\mathbf{S} = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$$

ordered by decreasing eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$. The fraction of variance explained by the first m components is $\sum_{i=1}^m \lambda_i / \sum_{j=1}^d \lambda_j$.

Linear Discriminant Analysis. LDA maximises the ratio of between-class to within-class scatter:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}.$$

The solution is given by the generalised eigenvectors of $\mathbf{S}_B \mathbf{v} = \lambda \mathbf{S}_W \mathbf{v}$. LDA can produce at most $C - 1$ discriminative directions.

6.2 Experiment 6.1 — PCA from Scratch

```
1  #!/usr/bin/env python3
2  """Week 6 - Principal Component Analysis."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import load_iris
7  from sklearn.preprocessing import StandardScaler
8
9  class PCAScratch:
10     def __init__(self, n_components=2):
11         self.n_components = n_components
12
13     def fit(self, X):
14         X = np.asarray(X, dtype=float)
15         self.mean_ = X.mean(axis=0)
16         X_c = X - self.mean_
17         cov = np.cov(X_c, rowvar=False)
18         eigvals, eigvecs = np.linalg.eigh(cov)
19         # sort descending
20         idx = np.argsort(eigvals)[::-1]
21         self.components_ = eigvecs[:, idx[:self.n_components]].T
22         self.explained_variance_ = eigvals[idx[:self.n_components]]
23         self.explained_variance_ratio_ = (
24             self.explained_variance_ / eigvals.sum()
25         )
26         return self
27
28     def transform(self, X):
29         return (X - self.mean_) @ self.components_.T
30
31     def fit_transform(self, X):
32         return self.fit(X).transform(X)
33
34     iris = load_iris()
35     X = StandardScaler().fit_transform(iris.data)
36     pca = PCAScratch(n_components=2).fit(X)
37     X_pca = pca.transform(X)
38
39     print("Explained variance ratio:", pca.explained_variance_ratio_.round(3))
40     print("Cumulative:", np.cumsum(pca.explained_variance_ratio_).round(3))
41
42     plt.figure(figsize=(7, 5))
43     for c, name in enumerate(iris.target_names):
44         plt.scatter(X_pca[iris.target == c, 0], X_pca[iris.target == c, 1],
45                     label=name, edgecolor='k')
46     plt.xlabel("PC1"); plt.ylabel("PC2")
47     plt.legend(); plt.title("PCA of Iris (scratch)")
48     plt.savefig("pca_iris.png", dpi=150)
49     plt.show()
```

6.3 Experiment 6.2 — LDA Comparison

```

1  #!/usr/bin/env python3
2  from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
3  from sklearn.datasets import load_iris
4  from sklearn.preprocessing import StandardScaler
5  import matplotlib.pyplot as plt
6
7  iris = load_iris()
8  X = StandardScaler().fit_transform(iris.data)
9  y = iris.target
10
11  lda = LinearDiscriminantAnalysis(n_components=2)
12  X_lda = lda.fit_transform(X, y)
13
14  plt.figure(figsize=(7, 5))
15  for c, name in enumerate(iris.target_names):
16  plt.scatter(X_lda[y == c, 0], X_lda[y == c, 1], label=name, edgecolor='k'
17  )
18  plt.xlabel("LD1"); plt.ylabel("LD2")
19  plt.legend(); plt.title("LDA of Iris")
20  plt.savefig("lda_iris.png", dpi=150)
21  plt.show()
22  print("LDA explained variance ratio:", lda.explained_variance_ratio_.
23  round(3))

```

Assignment Questions

1. **(Code)** Implement PCA from scratch. Project the Iris (and Digits) data onto the first two principal components and plot the result coloured by class.
2. **(Code)** Report the explained-variance ratio of the first five components for the Digits dataset. How many components are needed to retain 95% of the variance?
3. **(Code)** Apply LDA to Iris and compare the 2-D projection with PCA. Which method yields better class separation?
4. **(Conceptual)** Why does LDA require class labels while PCA does not? In what situations might PCA be preferable to LDA?

Expected Outputs

- Iris PCA typically shows ~73% + ~23% variance on the first two PCs; setosa is clearly separated.
- Digits often need 20–40 components to reach 95% cumulative variance.
- LDA projection of Iris usually shows even clearer class separation than PCA because it uses label information.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
PCA from scratch	Correct eigendecomposition, projection	30
Explained-variance analysis	Correct ratios and 95% threshold	20
LDA vs PCA visualization	Clear side-by-side comparison	25
Conceptual answer	Accurate distinction of supervised/unsupervised	15
Code quality	Clean, matches sklearn within tolerance	10
Total		100

Extra Experiment (Optional / Advanced)

Reconstruction error & image eigenfaces (optional). Using the Digits dataset, reconstruct a few sample images from the first $m = 10, 20, 40$ principal components. Compute the mean squared reconstruction error as a function of m and plot it. Visually inspect how the digits degrade.

```
1 from sklearn.decomposition import PCA
2 pca = PCA(n_components=40).fit(X)
3 X_hat = pca.inverse_transform(pca.transform(X))
4 mse = np.mean((X - X_hat)**2, axis=1)
5 print("Mean reconstruction MSE (40 PCs):", mse.mean().round(4))
6
```

6.4 Assignment Questions

Post-Lab Reflection & Discussion

1. How many principal components are needed to retain 95% of the variance on Digits? On Iris?
2. Why does LDA produce at most $C - 1$ meaningful directions?
3. In a face-recognition pipeline would you prefer PCA or LDA for the projection step? Justify.
4. Reflect on a situation where reducing dimensionality *hurts* downstream classification accuracy.

Part IV

Supervised Learning

LAB 7

SUPPORT VECTOR MACHINES

7.1 Objective

Learning Objectives

- Train linear and kernel SVMs using scikit-learn.
- Visualize margins and support vectors.
- Tune the hyper-parameters C and γ via cross-validation.

Pre-Lab Preparation

1. What is the geometric meaning of the *margin* of a linear classifier?
2. Write the soft-margin SVM primal objective (with slack variables) from your notes.
3. What is the role of the kernel trick? Name three common kernels.
4. Why does the number of support vectors matter for the sparsity of the solution?

Theory Notes

Hard-margin SVM. Maximise the geometric margin $2/\|\mathbf{w}\|$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$. **Soft-margin SVM** introduces slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i.$$

Large C penalises margin violations heavily (tends to over-fit); small C allows a wider margin.

Kernel SVM. Replace inner products $\mathbf{x}_i^\top \mathbf{x}_j$ by a positive-definite kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ (e.g., RBF $K = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$), which implicitly maps data into a higher-dimensional feature space.

7.2 Experiment 7.1 — Linear SVM and Margin Visualization

```
#!/usr/bin/env python3
```

```
2 """Week 7 - Support Vector Machines."""
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from sklearn.datasets import make_classification
7 from sklearn.svm import SVC
8 from sklearn.model_selection import train_test_split
9
10 X, y = make_classification(n_samples=100, n_features=2, n_redundant=0,
11 n_informative=2, n_clusters_per_class=1,
12 class_sep=1.2, random_state=42)
13 y = y % 2
14 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
15 random_state=0)
16
17 clf = SVC(kernel='linear', C=1.0).fit(X_tr, y_tr)
18 print("Train accuracy:", clf.score(X_tr, y_tr).round(3))
19 print("Test accuracy:", clf.score(X_te, y_te).round(3))
20 print("Number of support vectors:", clf.n_support_)
21
22 # Decision boundary + margin
23 def plot_svm(clf, X, y):
24     plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolor='k')
25     ax = plt.gca()
26     xlim, ylim = ax.get_xlim(), ax.get_ylim()
27     xx = np.linspace(xlim[0], xlim[1], 200)
28     yy = np.linspace(ylim[0], ylim[1], 200)
29     YY, XX = np.meshgrid(yy, xx)
30     xy = np.vstack([XX.ravel(), YY.ravel()]).T
31     Z = clf.decision_function(xy).reshape(XX.shape)
32     ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1],
33 alpha=0.7, linestyles=['--', '-', '--'])
34     ax.scatter(clf.support_vectors_[0], clf.support_vectors_[1],
35 s=120, facecolors='none', edgecolors='k', linewidths=1.5,
36 label='support vectors')
37     plt.legend()
38
39 plt.figure(figsize=(7, 5))
40 plot_svm(clf, X_tr, y_tr)
41 plt.title("Linear SVM (C=1)")
42 plt.savefig("svm_linear.png", dpi=150)
43 plt.show()
```

7.3 Experiment 7.2 — RBF Kernel and Hyper-parameter Grid

```
1 #!/usr/bin/env python3
2 from sklearn.svm import SVC
3 from sklearn.model_selection import GridSearchCV
4 from sklearn.datasets import load_iris
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.pipeline import Pipeline
7
8 iris = load_iris()
9 X, y = iris.data, iris.target
10
11 pipe = Pipeline([
12     ('scaler', StandardScaler()),
```

```

13 ('svc', SVC(kernel='rbf'))
14 ])
15 param_grid = {
16     'svc__C': [0.1, 1, 10, 100],
17     'svc__gamma': [0.01, 0.1, 1, 'scale']
18 }
19 grid = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs
20                       =-1)
21 grid.fit(X, y)
22 print("Best params:", grid.best_params_)
23 print("Best CV accuracy:", round(grid.best_score_, 3))

```

Assignment Questions

1. **(Code)** Train a linear SVM on a 2-D synthetic dataset. Plot the decision boundary, the margins, and highlight the support vectors.
2. **(Code)** Compare linear, polynomial and RBF kernels on the Iris dataset (standardized). Report test accuracy for each.
3. **(Code)** Perform a grid search over C and γ for an RBF SVM on the Digits dataset (or a subset). Report the best parameters and the corresponding cross-validation score.
4. **(Conceptual)** Explain the role of the soft-margin parameter C . What happens when $C \rightarrow 0$ and when $C \rightarrow \infty$?

Expected Outputs

- Linear SVM plot shows a solid decision line and two dashed margin lines; support vectors lie on or inside the margin.
- RBF kernel usually yields the highest accuracy on Iris among the three kernels.
- Grid search returns a sensible (C, γ) pair with CV accuracy typically > 0.95 on Iris and high on Digits.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Linear SVM visualization	Boundary, margins, support vectors correct	25
Kernel comparison	Clear table of accuracies	25
Grid-search experiment	Proper CV, reported best params & score	25
Conceptual answer	Correct interpretation of C	15
Code quality	Clean pipelines, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

Effect of C on the margin. Train linear SVMs with $C \in \{0.01, 0.1, 1, 10, 100\}$ on a 2-D dataset. For each C plot the decision boundary and count the number of support vectors. Observe how the margin shrinks and the number of SVs changes as C grows.

7.4 Assignment Questions

Post-Lab Reflection & Discussion

1. How many support vectors did the linear SVM use? What fraction of the training set is that?
2. Which kernel performed best on Iris? Was the difference statistically noticeable?
3. Explain in your own words why a very large γ in an RBF kernel can lead to over-fitting.
4. Reflect: when would you prefer an SVM over a Random Forest (or vice-versa) in a practical project?

LAB 8

DECISION TREES & ENSEMBLE METHODS

8.1 Objective

Learning Objectives

- Train and visualize Decision Trees.
- Understand impurity measures (Gini, entropy) and pruning.
- Build Random Forests and compare with a single tree.

Pre-Lab Preparation

1. Write the formulae for Gini impurity and entropy of a node.
2. What is the difference between pre-pruning and post-pruning?
3. How does a Random Forest inject randomness (bootstrap sampling + feature subsampling)?
4. Why do ensembles of trees usually outperform a single deep tree?

Theory Notes

Decision Trees. Recursively partition the feature space by choosing the split that maximises impurity reduction (information gain or Gini gain). Deep trees have low bias but high variance and easily over-fit.

Random Forests. Build B trees on bootstrap samples; at each split consider only a random subset of $\sqrt{d}$ (classification) or $d/3$ (regression) features. The final prediction is the majority vote (or average). This bagging + feature randomness dramatically reduces variance while keeping bias low.

8.2 Experiment 8.1 — Decision Tree and Feature Importance

```
1  #!/usr/bin/env python3
2  """Week 8 - Decision Trees and Random Forests."""
3
4  import matplotlib.pyplot as plt
5  from sklearn.datasets import load_iris
```

```
6 from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
7 from sklearn.model_selection import train_test_split
8 from sklearn.metrics import accuracy_score
9
10 iris = load_iris()
11 X, y = iris.data, iris.target
12 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
13     random_state=42, stratify=y)
14
15 tree = DecisionTreeClassifier(max_depth=3, random_state=0)
16 tree.fit(X_tr, y_tr)
17 print("Test accuracy:", round(accuracy_score(y_te, tree.predict(X_te)),
18     3))
19 print("Feature importances:", dict(zip(iris.feature_names, tree.
20     feature_importances_.round(3))))
21
22 plt.figure(figsize=(12, 6))
23 plot_tree(tree, feature_names=iris.feature_names, class_names=iris.
24     target_names,
25     filled=True, rounded=True, fontsize=9)
26 plt.title("Decision Tree (max_depth=3)")
27 plt.tight_layout()
28 plt.savefig("decision_tree.png", dpi=150)
29 plt.show()
30
31 print(export_text(tree, feature_names=list(iris.feature_names)))
```

Listing 8.1: Decision Tree and Feature Importance

8.3 Experiment 8.2 — Random Forest Comparison

```
1 #!/usr/bin/env python3
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.datasets import load_digits
4 from sklearn.model_selection import cross_val_score
5 import numpy as np
6
7 digits = load_digits()
8 X, y = digits.data, digits.target
9
10 tree = DecisionTreeClassifier(random_state=0)
11 rf = RandomForestClassifier(n_estimators=100, random_state=0, n_jobs=-1)
12
13 scores_tree = cross_val_score(tree, X, y, cv=5)
14 scores_rf = cross_val_score(rf, X, y, cv=5)
15 print(f"Single Tree CV accuracy: {scores_tree.mean():.3f} +/- {
16     scores_tree.std():.3f}")
17 print(f"Random Forest CV accuracy: {scores_rf.mean():.3f} +/- {scores_rf.
18     std():.3f}")
```

Listing 8.2: Random Forest Comparison

Assignment Questions

1. **(Code)** Train a Decision Tree on Iris with `max_depth` ranging from 1 to 6. Plot training and test accuracy versus depth (illustrate over-fitting).
2. **(Code)** Visualize the tree (depth ≤ 3) and list the feature importances.
3. **(Code)** Compare a single Decision Tree with a Random Forest (100 trees) on the Digits dataset using 5-fold cross-validation.
4. **(Conceptual)** Explain bagging and why Random Forests reduce variance compared with a single deep tree.

Expected Outputs

- Accuracy-vs-depth curve rises then plateaus or drops on the test set, illustrating over-fitting.
- Tree plot is readable; petal length/width dominate feature importances on Iris.
- Random Forest CV accuracy on Digits is substantially higher and more stable than a single tree.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Depth experiment & plots	Clear train/test curves showing overfit	25
Tree visualization & importances	Correct plot and numeric importances	25
RF vs Tree comparison	Proper CV, meaningful improvement shown	25
Conceptual answer	Correct explanation of bagging/variance	15
Code quality	Clean, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

Gradient Boosting comparison. Train a `HistGradientBoostingClassifier` (or `GradientBoostingClassifier`) on Digits and compare 5-fold CV accuracy and training time against the Random Forest from Experiment 8.2. Plot feature importances for both models side by side.

```

1  from sklearn.ensemble import HistGradientBoostingClassifier,
    RandomForestClassifier
2  from sklearn.datasets import load_digits
3  from sklearn.model_selection import cross_val_score
4  import time
5
6  X, y = load_digits(return_X_y=True)
7  for name, clf in [
8      ("RF-100", RandomForestClassifier(n_estimators=100, random_state=0,
9      n_jobs=-1)),
10     ("HGB", HistGradientBoostingClassifier(max_iter=100, random_state=0))
11 ]:
12     t0 = time.time()
13     scores = cross_val_score(clf, X, y, cv=5)
14     print(f"{name:8s}  acc={scores.mean():.3f}+/-{scores.std():.3f}  time
    = {time.time()-t0:.1f}s")

```

8.4 Assignment Questions

Post-Lab Reflection & Discussion

1. At what tree depth did you first observe a clear train/test accuracy gap? What does that tell you about model capacity?
2. Which features dominated the Iris tree? Does this agree with the pair-plot from Week 1?
3. Why does increasing the number of trees in a Random Forest rarely cause over-fitting, whereas increasing the depth of a single tree does?
4. Write a short reflection (5–8 sentences) on when you would prefer a single interpretable tree versus an ensemble.

LAB 9

INTRODUCTION TO NEURAL NETWORKS

9.1 Objective

Learning Objectives

- Implement a simple multi-layer perceptron (MLP) with NumPy or use scikit-learn / Keras.
- Train on a non-linearly separable problem and visualize the decision surface.
- Observe the effect of architecture and learning-rate choices.

Pre-Lab Preparation

1. Recall the Perceptron from Week 3. Why can a single-layer perceptron *not* solve the XOR problem?
2. Write the formulae for the sigmoid, tanh and ReLU activation functions. Sketch each.
3. What is back-propagation in one sentence?
4. List three hyper-parameters of an MLP that affect capacity and training dynamics.

Theory Notes

Multi-layer Perceptron. An MLP composes affine transformations with element-wise non-linearities:

$$\mathbf{h}^{(1)} = \sigma(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{h}^{(2)} = \sigma(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}), \quad \dots$$

ending in a softmax (classification) or linear (regression) output layer. Universal approximation theorems guarantee that a sufficiently wide single hidden layer can approximate any continuous function on a compact set; depth often yields more parameter-efficient representations.

Training. Parameters are optimised by (stochastic) gradient descent on a loss (cross-entropy or squared error). Gradients are computed by reverse-mode automatic differentiation (back-propagation). Practical concerns include learning-rate schedules, weight initialisation, regularisation (weight decay, dropout, early stopping) and batch normalisation.

9.2 Experiment 9.1 — MLP with scikit-learn

```
1  #!/usr/bin/env python3
2  """Week 9 - Multi-layer Perceptron."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_moons
7  from sklearn.model_selection import train_test_split
8  from sklearn.neural_network import MLPClassifier
9  from sklearn.preprocessing import StandardScaler
10
11 X, y = make_moons(n_samples=400, noise=0.2, random_state=42)
12 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
13     random_state=0)
14
15 scaler = StandardScaler()
16 X_tr = scaler.fit_transform(X_tr)
17 X_te = scaler.transform(X_te)
18
19 mlp = MLPClassifier(hidden_layer_sizes=(20, 10), activation='relu',
20     max_iter=1000, random_state=0)
21 mlp.fit(X_tr, y_tr)
22 print("Test accuracy:", round(mlp.score(X_te, y_te), 3))
23
24 # Decision surface
25 xx, yy = np.meshgrid(np.linspace(X_tr[:,0].min()-0.5, X_tr[:,0].max()+0.5, 200),
26     np.linspace(X_tr[:,1].min()-0.5, X_tr[:,1].max()+0.5, 200))
27 Z = mlp.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
28 plt.figure(figsize=(7, 5))
29 plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdBu)
30 plt.scatter(X_tr[:, 0], X_tr[:, 1], c=y_tr, edgecolor='k', cmap=plt.cm.
31     RdBu)
32 plt.title("MLP Decision Surface (make_moons)")
33 plt.savefig("mlp_moons.png", dpi=150)
34 plt.show()
```

Listing 9.1: MLP with scikit-learn

9.3 Experiment 9.2 — Learning Curves

```
1  #!/usr/bin/env python3
2  import matplotlib.pyplot as plt
3  from sklearn.neural_network import MLPClassifier
4  from sklearn.datasets import load_digits
5  from sklearn.model_selection import train_test_split
6  from sklearn.preprocessing import StandardScaler
7
8  digits = load_digits()
9  X, y = digits.data, digits.target
10 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
11     random_state=0, stratify=y)
12 scaler = StandardScaler()
13 X_tr = scaler.fit_transform(X_tr)
14 X_te = scaler.transform(X_te)
```

```

15 mlp = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300,
16 warm_start=True, random_state=0)
17 train_acc, test_acc = [], []
18 for i in range(1, 21):
19     mlp.max_iter = i * 15
20     mlp.fit(X_tr, y_tr)
21     train_acc.append(mlp.score(X_tr, y_tr))
22     test_acc.append(mlp.score(X_te, y_te))
23
24 plt.plot(range(15, 301, 15), train_acc, label='train')
25 plt.plot(range(15, 301, 15), test_acc, label='test')
26 plt.xlabel("Iterations"); plt.ylabel("Accuracy")
27 plt.legend(); plt.title("MLP Learning Curves (Digits)")
28 plt.savefig("mlp_learning_curves.png", dpi=150)
29 plt.show()

```

Listing 9.2: Learning Curves

Assignment Questions

1. **(Code)** Train an MLP on the two-moons dataset. Visualize the non-linear decision surface.
2. **(Code)** Experiment with different hidden-layer sizes (e.g., (5,), (20,), (50, 20)) and activation functions; report test accuracy.
3. **(Code)** Plot learning curves (train/test accuracy vs. iterations) on the Digits dataset.
4. **(Conceptual)** Why can a multi-layer network solve problems that a single perceptron cannot? What is the role of non-linear activation functions?

Expected Outputs

- MLP achieves high accuracy (> 0.90) on two-moons and produces a smooth curved decision boundary.
- Larger networks generally fit better but may over-fit if capacity is excessive relative to data size.
- Learning curves show rapid early improvement then plateau; a small gap between train and test indicates good generalization.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
MLP on moons + visualization	Correct non-linear boundary	25
Architecture experiments	Systematic comparison of sizes/activations	25
Learning curves	Clear train/test plots	25
Conceptual answer	Correct explanation of non-linearity & depth	15
Code quality	Clean, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

Mini NumPy MLP from scratch (XOR). Implement a two-layer network (2-2-1) with sigmoid activations and train it on the XOR truth table using gradient descent. Plot the loss curve and verify

that the network learns the correct outputs.

```
1 import numpy as np
2
3 def sigmoid(z): return 1/(1+np.exp(-z))
4 def sigmoid_grad(z): s=sigmoid(z); return s*(1-s)
5
6 X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
7 y = np.array([[0],[1],[1],[0]], dtype=float)
8
9 rng = np.random.RandomState(0)
10 W1, b1 = rng.randn(2,2)*0.5, np.zeros((1,2))
11 W2, b2 = rng.randn(2,1)*0.5, np.zeros((1,1))
12 lr, losses = 0.5, []
13
14 for epoch in range(5000):
15     # forward
16     z1 = X@W1 + b1; a1 = sigmoid(z1)
17     z2 = a1@W2 + b2; a2 = sigmoid(z2)
18     loss = np.mean((a2-y)**2); losses.append(loss)
19     # backward
20     dz2 = (a2-y)*sigmoid_grad(z2)
21     dW2 = a1.T@dz2; db2 = dz2.sum(axis=0, keepdims=True)
22     dz1 = (dz2@W2.T)*sigmoid_grad(z1)
23     dW1 = X.T@dz1; db1 = dz1.sum(axis=0, keepdims=True)
24     W2 -= lr*dW2; b2 -= lr*db2; W1 -= lr*dW1; b1 -= lr*db1
25
26 print("Final predictions:\n", a2.round(3))
27 print("Final loss:", round(losses[-1], 5))
28
```

9.4 Assignment Questions

Post-Lab Reflection & Discussion

1. How does the decision surface of the MLP on two-moons differ from that of a linear classifier?
2. Did increasing the hidden-layer size always improve test accuracy? If not, why?
3. What early-stopping rule would you use based on the learning curves you plotted?
4. Write a short reflection (5–8 sentences) on the practical challenges of training neural networks (learning rate, initialisation, over-fitting).

LAB 10

MODEL EVALUATION & SELECTION

10.1 Objective

Learning Objectives

- Compute and interpret confusion matrices, precision, recall, F1, ROC curves.
- Perform k -fold and stratified cross-validation.
- Compare multiple classifiers under a fair evaluation protocol.

Pre-Lab Preparation

1. Define true positive, false positive, true negative and false negative.
2. Write the formulae for precision, recall and F1-score.
3. What does the area under the ROC curve (AUC) represent?
4. Why is stratified k -fold preferred over ordinary k -fold for imbalanced problems?

Theory Notes

Confusion Matrix & Derived Metrics. For a binary problem the confusion matrix counts TP, FP, TN, FN. Precision = $TP/(TP + FP)$, Recall (sensitivity) = $TP/(TP + FN)$, F1 is their harmonic mean. Accuracy can be misleading under class imbalance.

ROC & PR Curves. The ROC curve plots TPR vs. FPR as the decision threshold varies; AUC summarises ranking quality. Precision-Recall curves are more informative when the positive class is rare.

Cross-Validation. k -fold CV partitions the data into k folds, trains on $k - 1$ and evaluates on the held-out fold, rotating through all folds. Stratified CV preserves class proportions. Nested CV is required when hyper-parameters are tuned, to avoid optimistic bias.

10.2 Experiment 10.1 — Comprehensive Metrics

```
#!/usr/bin/env python3
```

```
2  """Week 10 - Evaluation metrics and ROC."""
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import load_breast_cancer
7  from sklearn.model_selection import train_test_split, cross_val_score,
   StratifiedKFold
8  from sklearn.preprocessing import StandardScaler
9  from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import (confusion_matrix, classification_report,
11 roc_curve, auc, RocCurveDisplay)
12
13 data = load_breast_cancer()
14 X, y = data.data, data.target
15 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
16     random_state=42, stratify=y)
17
18 scaler = StandardScaler()
19 X_tr = scaler.fit_transform(X_tr)
20 X_te = scaler.transform(X_te)
21
22 clf = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
23 y_pred = clf.predict(X_te)
24 y_score = clf.predict_proba(X_te)[:, 1]
25
26 print("Confusion matrix:\n", confusion_matrix(y_te, y_pred))
27 print(classification_report(y_te, y_pred, target_names=data.target_names)
28 )
29
30 fpr, tpr, _ = roc_curve(y_te, y_score)
31 roc_auc = auc(fpr, tpr)
32 plt.figure(figsize=(6, 5))
33 plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
34 plt.plot([0, 1], [0, 1], 'k--')
35 plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
36 plt.title("ROC Curve - Breast Cancer")
37 plt.legend(); plt.grid(True, alpha=0.3)
38 plt.savefig("roc_curve.png", dpi=150)
39 plt.show()
```

Listing 10.1: Comprehensive Metrics

10.3 Experiment 10.2 — Cross-Validation Comparison

```
1  #!/usr/bin/env python3
2  from sklearn.datasets import load_wine
3  from sklearn.model_selection import cross_val_score, StratifiedKFold
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.pipeline import Pipeline
6  from sklearn.neighbors import KNeighborsClassifier
7  from sklearn.svm import SVC
8  from sklearn.ensemble import RandomForestClassifier
9  from sklearn.naive_bayes import GaussianNB
10 import numpy as np
11
12 wine = load_wine()
13 X, y = wine.data, wine.target
```

```

14 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
15
16 models = {
17     "k-NN (k=5)": KNeighborsClassifier(n_neighbors=5),
18     "RBF SVM": SVC(kernel='rbf'),
19     "Random Forest": RandomForestClassifier(n_estimators=100, random_state
20     =0),
21     "Gaussian NB": GaussianNB()
22 }
23
24 for name, model in models.items():
25     pipe = Pipeline([('scaler', StandardScaler()), ('clf', model)])
26     scores = cross_val_score(pipe, X, y, cv=cv, scoring='accuracy')
27     print(f"{name:20s} {scores.mean():.3f} +/- {scores.std():.3f}")

```

Listing 10.2: Cross-Validation Comparison

Assignment Questions

1. **(Code)** On the Breast Cancer dataset train a classifier of your choice and produce a confusion matrix, classification report, and ROC curve with AUC.
2. **(Code)** Compare at least four different classifiers on the Wine (or Digits) dataset using stratified 5-fold cross-validation. Present results in a table.
3. **(Code)** Plot ROC curves for two classifiers on the same axes and discuss which one is preferable under different operating points.
4. **(Conceptual)** Explain the difference between accuracy, precision, recall and F1. When is accuracy a misleading metric?

Expected Outputs

- Confusion matrix and classification report show per-class precision/recall; ROC AUC for Breast Cancer is typically > 0.95.
- Cross-validation table ranks the classifiers; Random Forest or SVM often lead on Wine/Digits.
- Overlaid ROC curves make the trade-off between true-positive and false-positive rates visible.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Metrics & ROC on one dataset	Correct matrix, report, ROC+AUC	25
Multi-classifier CV comparison	Fair protocol, clear table	30
ROC comparison discussion	Insightful operating-point analysis	20
Conceptual answer	Accurate metric definitions & pitfalls	15
Code quality	Clean pipelines, reproducible	10
Total		100

Extra Experiment (Optional / Advanced)

McNemar's test for classifier comparison. Train two classifiers on the same train/test split of Breast Cancer. Construct the contingency table of their disagreements and apply McNemar's test to

decide whether the difference in accuracy is statistically significant.

```
1  from sklearn.datasets import load_breast_cancer
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.linear_model import LogisticRegression
5  from sklearn.ensemble import RandomForestClassifier
6  from statsmodels.stats.contingency_tables import mcnemar
7  import numpy as np
8
9  X, y = load_breast_cancer(return_X_y=True)
10 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
11     random_state=0, stratify=y)
12 sc = StandardScaler()
13 X_tr, X_te = sc.fit_transform(X_tr), sc.transform(X_te)
14 p1 = LogisticRegression(max_iter=1000).fit(X_tr, y_tr).predict(X_te)
15 p2 = RandomForestClassifier(n_estimators=100, random_state=0).fit(X_tr
16     , y_tr).predict(X_te)
17
18 # Contingency: both correct, 1 only, 2 only, both wrong
19 a = np.sum((p1==y_te) & (p2==y_te))
20 b = np.sum((p1==y_te) & (p2!=y_te))
21 c = np.sum((p1!=y_te) & (p2==y_te))
22 d = np.sum((p1!=y_te) & (p2!=y_te))
23 table = [[a, b], [c, d]]
24 print("Contingency table:", table)
25 print(mcnemar(table, exact=False, correction=True))
```

10.4 Assignment Questions

Post-Lab Reflection & Discussion

1. For the Breast Cancer task, is accuracy a sufficient metric? Why or why not?
2. Looking at the ROC curves, at which operating point would you set the threshold if false negatives are twice as costly as false positives?
3. Did any classifier dominate all others across every metric, or were there trade-offs?
4. Write a short reflection (5–8 sentences) on the importance of a rigorous evaluation protocol before deploying a model.

Part V

Capstone Project

LAB 11

CAPSTONE — COMPLETE PATTERN RECOGNITION PIPELINE

11.1 Objective

Learning Objectives

- Design and execute an end-to-end pattern recognition system on a chosen dataset.
- Integrate preprocessing, feature extraction/selection, model training, hyper-parameter tuning and rigorous evaluation.
- Deliver a professional technical report and reproducible code.

Pre-Lab Preparation

1. Choose your dataset (from the approved list or with instructor permission) and write a one-paragraph problem statement.
2. Sketch the planned pipeline on paper: preprocessing → features → models → evaluation.
3. List at least three model families you will compare and the hyper-parameters you will tune.
4. Prepare a clean Python environment with all required libraries installed and tested.

Theory Notes

End-to-End Design Principles. A production-grade pattern recognition system must address: (1) data quality and leakage prevention, (2) appropriate feature representation, (3) model selection under a nested evaluation protocol, (4) uncertainty / calibration awareness, and (5) interpretability or explainability requirements of the application domain. The bias-variance trade-off, the no-free-lunch theorem, and Occam's razor remain guiding principles when comparing complex models against simpler baselines.

Ethical & Legal Warning

The capstone must be performed on one of the approved datasets (or an instructor-approved alternative). All results must be reproducible from the submitted code.

11.2 Suggested Pipeline

1. **Problem definition & data understanding** — choose dataset, explore distributions, class balance, missing values.
2. **Preprocessing** — cleaning, scaling, encoding, train/validation/test split (or nested CV).
3. **Feature engineering / selection** — PCA, mutual information, recursive feature elimination, domain features.
4. **Model zoo** — train at least three distinct families (e.g., k -NN, SVM, Random Forest, MLP).
5. **Hyper-parameter optimization** — grid or random search with cross-validation.
6. **Final evaluation** — confusion matrix, ROC/PR curves, statistical comparison.
7. **Error analysis & discussion** — failure cases, limitations, possible improvements.

11.3 Capstone Deliverables

- Executive summary (≤ 1 page).
- Technical report (4–8 pages) covering the pipeline above.
- Fully reproducible Python code (script or well-documented notebook).
- Figures: data exploration, learning curves, ROC, confusion matrices, feature importance, etc.

11.4 Sample Report Outline

1. Introduction & Problem Statement
2. Dataset Description & Exploratory Analysis
3. Methodology (preprocessing, features, models, tuning)
4. Experimental Results
5. Discussion & Error Analysis
6. Conclusions & Future Work
7. Appendix (extra figures, full hyper-parameter tables)

Assignment Questions

1. **(Practical)** Select an approved dataset and implement a complete PR pipeline following the seven steps listed above.
2. **(Report)** Write a professional technical report (structure given) that a colleague could reproduce.
3. **(Code)** Submit clean, commented, runnable code that produces all reported numbers and figures.
4. **(Reflection)** Identify the single most important design choice you made and justify it with evidence from your experiments.

Expected Outputs

- Pipeline runs end-to-end without manual intervention (apart from configuration).
- At least three model families are fairly compared under the same evaluation protocol.

- Report contains quantitative results, clear figures, and a thoughtful discussion of limitations.
- Code reproduces the key tables and plots when executed on a clean environment.

Weekly Grading Rubric (Total: 100 points)

Criterion	Description	Pts
Pipeline completeness	All major stages present and justified	20
Experimental rigor	Fair comparison, proper CV / hold-out	25
Quality of report	Structure, clarity, figures, discussion	30
Code reproducibility	Runs cleanly, produces reported results	15
Insight & reflection	Thoughtful analysis of design choices	10
Total		100

Extra Experiment (Optional / Advanced)

Optional advanced extensions (choose one).

- **Calibration:** After selecting your best model, plot a reliability diagram and compute the Expected Calibration Error (ECE). Apply Platt scaling or isotonic regression and show the improvement.
- **Explainability:** Use permutation feature importance or SHAP values to explain individual predictions of your best model; include 2–3 illustrative examples in the report.
- **Cost-sensitive learning:** Define a realistic cost matrix for your problem and optimise the decision threshold (or train a cost-sensitive classifier) to minimise expected cost rather than error rate.

11.5 Assignment Questions

Post-Lab Reflection & Discussion

1. What was the single most impactful design choice in your pipeline, and what evidence supports that claim?
2. Which model family performed best, and did a simple baseline come surprisingly close?
3. If you had two more weeks, what experiment would you run next?
4. Submit the full report, code repository (or zip), and a short oral/demo summary if required by the instructor.

Part VI

Appendixes

APPENDIX A

LINUX BASICS

A.1 What is Linux?

Linux is a free and open-source operating system based on the Linux kernel. It is widely used in servers, cloud computing, embedded systems, and cybersecurity.

A.2 Why Learn Linux?

- Most web servers run on Linux.
- Kali Linux is built for penetration testing.
- Powerful command-line interface (CLI).
- Highly secure and stable.
- Preferred operating system for cybersecurity professionals.

A.3 Linux Architecture

Linux consists of the following components:

- Kernel
- Shell
- System Libraries
- System Utilities
- User Applications.

A.4 Linux Basics Practical

The next basic Linux commands are essential for navigation, system identification, and everyday system administration. They form the foundation for advanced Linux and cybersecurity tasks.

A.4.1 Experiment A.1 — Print Working Directory

```
1 # The 'pwd' (Print Working Directory) command displays the absolute path of
   the current directory.
2 $ pwd
```

Listing A.1: Print Working Directory

A.4.2 Experiment A.2 — Check Current User

```
1 # The 'whoami' command shows the name of the current user.
2 $ whoami
```

Listing A.2: Check Current User

A.4.3 Experiment A.3 — Display System Information

```
1 # The 'uname -a' command displays the kernel version, machine architecture,
   hostname, and operating system details.
2 $ uname -a
```

Listing A.3: Display System Information

A.4.4 Experiment A.4 — Display Hostname

```
1 # Displays the hostname of the system.
2 $ hostname
```

Listing A.4: Display Hostname

A.4.5 Experiment A.5 — Display Logged-in Users

```
1 # Displays information about users currently logged into the system..
2 $ who
```

Listing A.5: Display Logged-in Users

A.4.6 Experiment A.6 — View Command History

```
1 # Displays the list of previously executed commands.
2 $ history
```

Listing A.6: View Command History

A.4.7 Experiment A.7 — List Files and Directories

```
1 # Displays the files and directories in the current directory..
2 $ ls
```

Listing A.7: List Files and Directories

APPENDIX B

REQUIRED PYTHON LIBRARIES

Library	Primary Use
numpy / scipy	Numerical computing, linear algebra
matplotlib / seaborn	Visualization
scikit-learn	Classifiers, metrics, preprocessing, CV
pandas	Tabular data handling
jupyter	Interactive experimentation

Table B.1: Core libraries used throughout the labs

APPENDIX C

SUGGESTED DATASETS

Dataset	Source	Typical Use
Iris	sklearn	Weeks 1–4, 6
Wine	sklearn	Weeks 1, 5, 10
Breast Cancer	sklearn	Weeks 1, 10
Digits	sklearn	Weeks 4, 6–9
make_moons / blobs	sklearn	Weeks 3, 5, 9
MNIST (subset)	OpenML / Keras	Capstone option

Table C.1: Recommended datasets for each stage of the course

APPENDIX D

FURTHER READING

- R. O. Duda, P. E. Hart, D. G. Stork — *Pattern Classification* (2nd ed.).
- C. M. Bishop — *Pattern Recognition and Machine Learning*.
- scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html
- A. Geron — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*.

ACKNOWLEDGMENTS

Practically none of the ideas in this manual are original. My greatest debt is to *W. Conklin et al.* [2], *W. Du* [3], and *B. M. Whitman and H. Mattord* [4] for writing their brilliant and influential textbooks. It is a pleasure to thank *Sinai University* for giving me the opportunity to teach a course from this manual afterwards.

BIBLIOGRAPHY

- [1] M. CIAMPA, *Security Awareness: Applying Practical Cybersecurity in Your World*, Cengage Learning, Inc., 6th ed., 2024.
- [2] W. CONKLIN, G. WHITE, C. COTHREN, R. DAVIS, AND D. WILLIAMS, *Principles of Computer Security*, McGraw Hill, 5th ed., 2022.
- [3] W. DU, *Computer Security: A Hands-on Approach*, Independently published, 2nd ed., 2019.
- [4] M. WHITMAN AND H. MATTORD, *Principles of Information Security*, Cengage Learning, Inc., 7th ed., 2022.